IPBeja
Instituto Politécnico de Beja

Escola Superior de Tecnologia e Gestão

Mestrado em Internet das Coisas

Relatório de Projeto

Integração de Sistemas

João Manuel Rafael Pica, Vasco Miguel Guerreiro Gomes

Beja, 7 de Dezembro de 2024

INSTITUTO POLITÉCNICO DE BEJA

Escola Superior de Tecnologia e Gestão

Mestrado em Internet das Coisas

Relatório de Projeto

Integração de Sistemas

João Manuel Rafael Pica, Vasco Miguel Guerreiro Gomes

Orientado por :

Doutor João Carlos da Silva Martins & Doutor José Jasnau Caeiro, IPBeja

Relatório de Projeto

2024

Índice

Índice **i**

Índice de Figuras **v**

1 Introdução **1**
1.1 Enquadramento 1
1.2 Objetivos 1
1.3 Organização do relatório 2

2 Estado da Arte **3**
2.1 Parkopedia - Análise Visual 4
2.1.1 Tarefas Executadas 4
2.1.2 Arquitetura e Tecnologias 4
2.1.3 Interface 5
2.2 ParkWhiz - Análise Visual 5
2.2.1 Tarefas Executadas 6
2.2.2 Arquitetura e Tecnologias 6
2.2.3 Interface 6
2.3 *Low Cost Smart Parking System with IoT* - Análise Física 8
2.3.1 Arquitetura e Tecnologias 8
2.3.2 Funcionamento Geral 8
2.4 *IoT Based Cost Effective Car Parking Management for Urban Area* - Análise Física 10
2.4.1 Arquitetura e Tecnologias 10
2.4.2 Funcionamento Geral 11

3 Análise do Sistema **13**
3.1 Casos de Uso 13
3.1.1 Procurar Vagas de Estacionamento 13
3.1.2 Reservar Lugar para Necessidades Especiais 14
3.1.3 Abrir Cancela do Lugar Reservado 15

3.1.4 Analisar a plataforma 15
3.2 Requisitos Funcionais 15
3.2.1 Autenticação e Registo 15
3.2.2 Pesquisa e Reserva de Lugares 16
3.2.3 Atualização em Tempo Real 16
3.2.4 Controle de Acesso 16
3.3 Requisitos Não Funcionais 16
3.3.1 Desempenho 16
3.3.2 Segurança 16
3.3.3 Usabilidade e Compatibilidade 16
3.3.4 Escalabilidade 16
3.3.5 Confiabilidade 17

4 Arquiteturas do Sistema 19
4.1 Base de Dados 21
4.1.1 Login 21
4.1.2 UserToken 22
4.1.3 User 22
4.1.4 Park 22
4.1.5 Spot 23
4.1.6 Vehicle 23
4.1.7 Reserve 23
4.2 Desenho da Arquitetura do Sistema 24
4.2.1 Utilizador 24
4.2.2 Interface *Web* 24
4.2.3 *Framework de APIs* 25
4.2.4 Base de Dados 25
4.2.5 *Broker MQTT* e *Scripts* Adjacentes 25
4.2.6 Micro-controladores 25
4.2.7 Sensores e Atuadores 25

5 Implementação do Sistema 27
5.1 Lista da tecnologias utilizadas 28
5.1.1 Hardware 28
5.1.2 Software e Bibliotecas 28
5.2 Funcionamento do Sistema 29
5.2.1 1a. Interação Utilizador - Interface *Web* 29
5.2.2 2a. Interação Interface *Web* - *API Framework* 29
5.2.3 3a. Interação *API Framework* - Base de Dados 30
5.2.4 1b. Interação *Python Scripts* - *Broker MQTT* 30

5.2.5 2b. Interação *Python Scripts* - Base de Dados 31
5.2.6 3b. Interação *Pi Camera* - *Python Scripts* 31
5.2.7 4b. Interação *API Framework* - *Python Scripts* 31
5.2.8 1d. & 2d. Interação Sensores - *Scripting* - Motor Servo 32
5.2.9 3d. Interação Sensores - *Scripting* 32
5.2.10 4d. & 5d. Interação *Scripting* - *Broker MQTT* 33
5.3 SSH & Wireguard 33

6 Testes ao Sistema **35**

7 Conclusões **37**

Bibliografia **39**

Índice de Figuras

2.1 Interfaces Gráficas - Parkopedia Parking 4
2.2 Interfaces Gráficas - Parkwhiz 5
2.3 Arquitetura Proposta Artigo 1 8
2.4 Arquitetura Proposta Artigo 2 10

4.1 Desenho do Parque de Estacionamento Interno 19
4.2 Desenho do Parque de Estacionamento Externo 20
4.3 Desenho da Arquitetura da Base de Dados 21
4.4 Desenho da Arquitetura - Abstrato 24

5.1 Desenho da Arquitetura com Recurso às Tecnologias 27

Capítulo 1

Introdução

1.1 Enquadramento

No cenário atual, a crescente urbanização e a alta densidade populacional têm resultado em um aumento significativo no tráfego urbano e na dificuldade de encontrar vagas de estacionamento disponíveis. Para enfrentar esse desafio, várias soluções têm sido desenvolvidas para facilitar a busca por lugares de estacionamento, com o objetivo de melhorar a eficiência e a experiência dos utilizadores.

Uma dessas soluções é o sistema ***ParkLookUp***. O seu objetivo principal é simplificar a procura por vagas em estacionamentos utilizando mapas que destacam a disponibilidade em tempo real. Além disso, o sistema permite que utilizadores com necessidades especiais, como deficiências ou demandas de carga e descarga, se registem na plataforma e reservem vagas adaptadas às suas necessidades.

1.2 Objetivos

Assim, o sistema pretende solucionar os seguintes problemas:

- **Disponibilidade de estacionamento:** A dificuldade para encontrar vagas de estacionamento disponíveis é um problema frequente em áreas urbanas densamente povoadas. **O sistema oferece informações atualizadas em tempo real sobre a disponibilidade de vagas**.

- **Acessibilidade para utilizadores com necessidades especiais:** Utilizadores com deficiências ou outras necessidades especiais frequentemente enfrentam desafios adicionais ao encontrar um estacionamento adequado. **O sistema oferece recursos específicos para assegurar que esses utilizadores tenham uma experiência de estacionamento acessível e sem complicações**.

Ao tratar desses problemas, o sistema ***ParkLookUp*** pretende aprimorar a experiência de estacionamento, tornando-a mais conveniente, acessível e eficiente para todos os utilizadores, além de beneficiar o meio ambiente.

1.3 Organização do relatório

Este relatório encontra-se estruturado em sete capítulos. Cada capítulo aborda aspetos específicos relacionados à elaboração do sistema. No seguimento, apresentamos uma breve descrição de cada capítulo:

1. **Estado de Arte:** Apresentação e análise de sistemas semelhantes ao ***ParkLookUp***;
2. **Análise do Sistema:** Analise do sistema explorando casos de uso dos utilizadores e requisitos funcionais e não funcionais;
3. **Arquiteturas do Sistema:** Desenvolvimento do desenho do sistema, apresentando as propostas das interfaces *web* e arquitetura para o sistema;
4. **Implementação e Testes do Sistema:** Explicitar todas as tecnologias e abordagens seguidas para implementar o projeto;
5. **Conclusão:** Conclusão dos temas discutidos e aperfeiçoados ao longo do desenvolvimento do projeto;
6. **Bibliografia:** Listagem de todas as referências utilizadas na elaboração do relatório.

Capítulo 2

Estado da Arte

Para entender melhor o tema, foi realizada uma busca por sistemas semelhantes ao proposto neste projeto. Essa busca não permitiu apenas encontrar abordagens já exploradas por outros sistemas, mas também identificar problemas enfrentados.

Deste modo, foram analisados sistemas semelhantes em duas vertentes:

- **Visual -** Aqui pretende-se analisar os sistemas do ponto de vista de interfaces *Web*. Os sistemas aqui analisados denominam-se de ***Parkopedia*** [Par23a] e ***ParkWhiz*** [Par23c].

- **Física -** Os sensores, os micro-controladores, os sistemas de comunicação, entre outros, foram o foco nesta análise. Os sistemas aqui analisados denominam-se de ***Low Cost Smart Parking System with IoT*** [C S20] e ***IoT Based Cost Effective Car Parking Management for Urban Area*** [Pey+21].

Toda a análise foi elaborada com base na opinião de ambos os alunos, podendo alguns dos pontos não coincidirem com a análise de outros pesquisadores.

2.1 Parkopedia - Análise Visual

O sistema ***Parkopedia*** tem como objetivo ajudar os utilizadores a encontrar lugares de estacionamento próximos do seu destino. Isso permite economizar tempo e combustível, reduzindo também as emissões de CO2 por veículo. [Par23a]

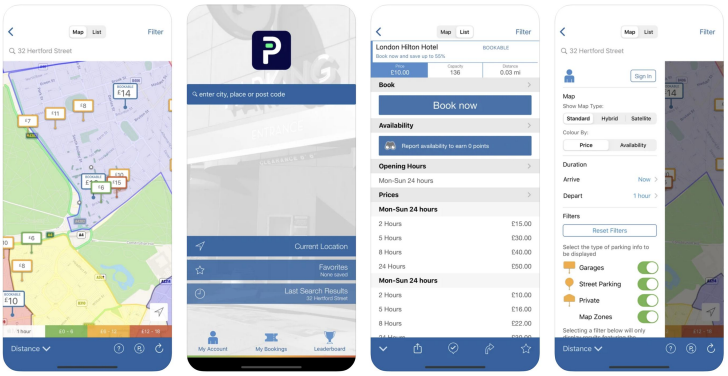

Figura 2.1: Interfaces Gráficas - Parkopedia Parking

2.1.1 Tarefas Executadas

Os utilizadores podem realizar várias tarefas através da aplicação [Par23a]:

- **Encontrar Estacionamento**: Localizar lugares de estacionamento próximos ao destino.
- **Pré-Reserva do Lugar**: Reservar o lugar de estacionamento online antecipadamente.
- **Consultar Status da Reserva**: Verificar o status da reserva feita.
- **Realizar Pagamento da Reserva**: Efetuar o pagamento do estacionamento online.

2.1.2 Arquitetura e Tecnologias

Embora detalhes específicos não sejam fornecidos pela empresa, a aplicação utiliza tecnologias comuns em sistemas similares, como:

- Plataformas baseadas em *cloud* [Mic24].
- GPS para localização [Nat23].
- Sensores IoT [Zip23] para monitorizar a disponibilidade dos lugares.
- Sistemas de pagamentos online.

A combinação dessas tecnologias permite fornecer informações em tempo real sobre a disponibilidade e preços dos estacionamentos.

2.1.3 Interface

A aplicação é projetada para ser amigável, facilitando tarefas como navegação em tempo real e reservas de lugares, especialmente na versão móvel. O *website* oferece informações detalhadas sobre os estacionamentos. Para funcionar, utiliza uma API que facilita conexões e interações com outras aplicações [Par23b].

2.2 ParkWhiz - Análise Visual

A aplicação ***ParkWhiz*** visa revolucionar a experiência de estacionamento, permitindo aos utilizadores procurar, comparar e reservar lugares de estacionamento em qualquer lugar e a qualquer momento. Isso facilita o planeamento antecipado de viagens ou a procura de lugares imediatos. [Par23c]

Figura 2.2: Interfaces Gráficas - Parkwhiz

2.2.1 Tarefas Executadas

Os utilizadores podem realizar várias tarefas através da aplicação [Par23e]:

- **Pesquisar Vagas de Estacionamento**: Busca por vagas disponíveis em áreas específicas.
- **Comparar Preços**: Comparação de preços entre diferentes locais de estacionamento.
- **Reservar Vaga**: Reserva de vagas com alguns cliques.
- **Pagamento Online**: Transações seguras de pagamento online.
- **Aceder ao Passe de Estacionamento Móvel**: Recebimento e acesso ao passe de estacionamento no dispositivo móvel.
- **Planeamento Antecipado**: Reserva de estacionamento com antecedência.
- **Economizar com Descontos**: Aproveitamento de descontos nas tarifas de estacionamento.
- **Aceder ao Histórico de Reservas:** Gestão e visualização do histórico de reservas.

2.2.2 Arquitetura e Tecnologias

Embora os detalhes específicos não sejam fornecidos pela empresa, a aplicação utiliza padrões comuns, como:

- Plataformas baseadas em *cloud* [Mic24].
- GPS para localização [Nat23].
- Sensores IoT [Zip23] para monitorizar disponibilidade.
- Sistemas de pagamentos online.

A combinação dessas tecnologias fornece informações em tempo real sobre disponibilidade e preços de estacionamento.

2.2.3 Interface

A interface é amigável e proporciona uma boa experiência de utilização [Par23d]:

- **Página Inicial**: Barra de pesquisa acessível, permitindo inserir locais específicos ou usar o GPS [Nat23] para encontrar estacionamentos próximos.

- **Resultados de Pesquisa**: Informações claras sobre disponibilidade, preços e descontos.
- **Processos de Reservas**: Simples, com opções de pagamento seguras integradas.
- **Detalhes de Reservas**: Visualização dos detalhes das reservas e acesso aos passes de estacionamento móvel.

2.3 *Low Cost Smart Parking System with IoT* - Análise Física

Este artigo apresenta uma arquitetura para um sistema de estacionamento inteligente de baixo custo que atualiza constantemente o estado de ocupação dos lugares de estacionamento [C S20].

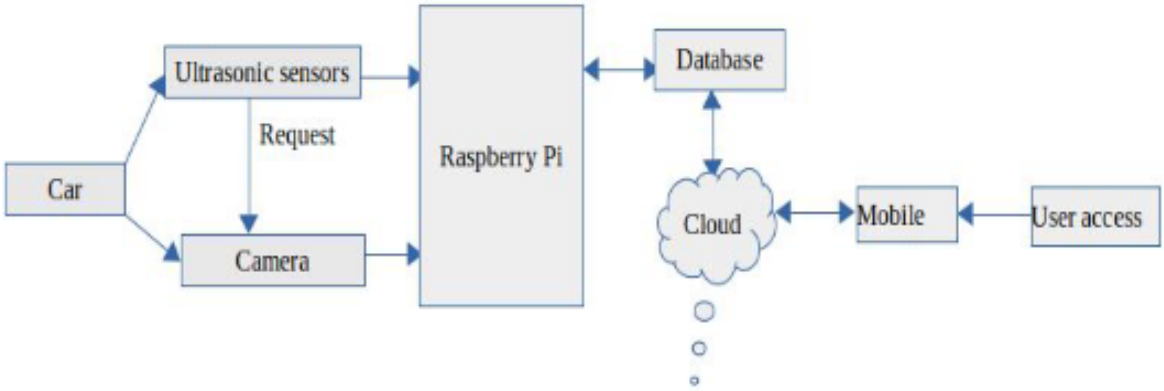

Figura 2.3: Arquitetura Proposta Artigo 1

2.3.1 Arquitetura e Tecnologias

A arquitetura proposta utiliza os seguintes componentes:

- **Sensores Ultrassónicos HC-SR04**: Detectam a ocupação dos lugares de estacionamento [Dig24].
- **Câmara**: Captura imagens dos lugares de estacionamento [Ras24b].
- **Raspberry Pi 3B**: Processa as imagens capturadas e envia-as para a nuvem [Ras24a].
- ***Google Cloud Vision API***: Realiza o processamento de imagem e a detecção da ocupação dos lugares [Goo24b].
- ***Cloud (Microsoft Azure)***: Armazena os dados de ocupação e facilita o acesso aos utilizadores através de dispositivos móveis [Mic24].

2.3.2 Funcionamento Geral

- **Deteção da Ocupação**: Sensores ultrassónicos e uma câmara capturam o estado de ocupação dos lugares [Dig24].

- **Processamento de Imagem**: Imagens capturadas são enviadas para o *Raspberry Pi 3B* [Ras24a], que as transmite para a *Google Cloud Vision API* [Goo24b].
- **Atualização de Dados**: A *Google Cloud Vision API* [Goo24b] processa as imagens e determina a ocupação dos lugares.
- **Armazenamento e Acesso**: A informação é enviada para uma *cloud (Microsoft Azure)* [Mic24], onde os dados são armazenados e disponibilizados para acesso por dispositivos móveis.

2.4 *IoT Based Cost Effective Car Parking Management for Urban Area* - Análise Física

Este sistema visa monitorizar todos os lugares de um parque de estacionamento, fornecendo informações sobre a ocupação e controlando o acesso ao parque através de uma cancela operada por um motor servo. [Pey+21]

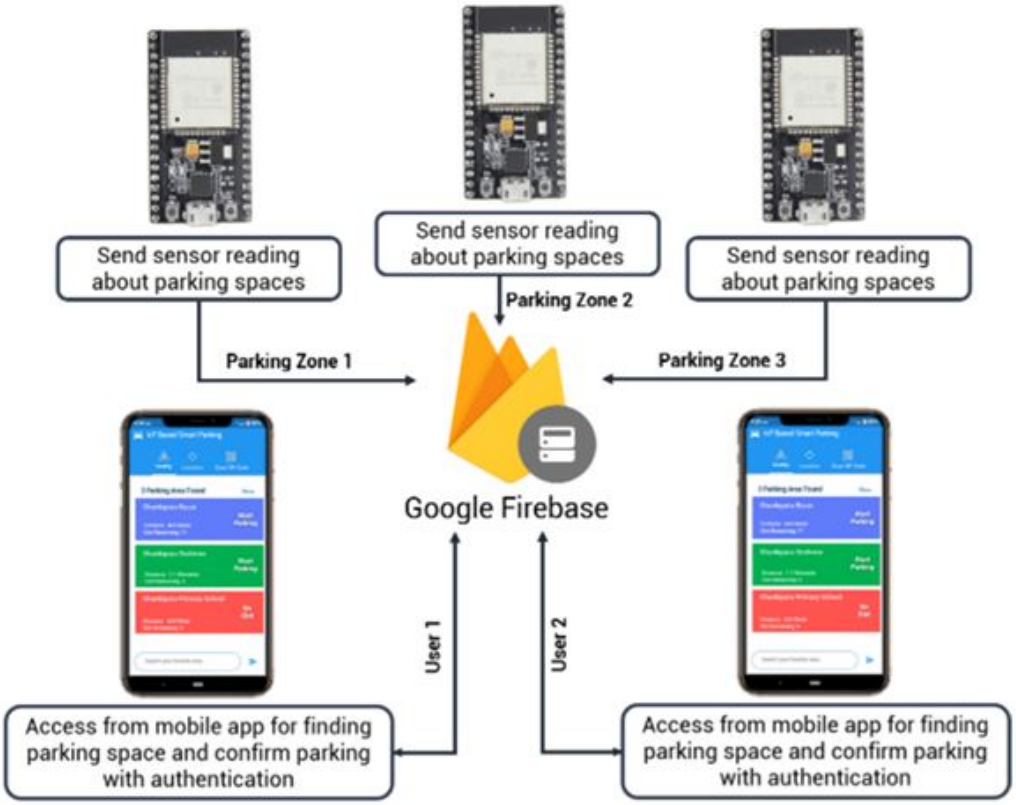

Figura 2.4: Arquitetura Proposta Artigo 2

2.4.1 Arquitetura e Tecnologias

A arquitetura do sistema é composta por componentes de hardware e software mais complexos, incluindo:

- **ESP32**: Três unidades de ESP32 são responsáveis por recolher informações de cada lugar de estacionamento [Esp24a].
- **Motor Servo**: Opera a cancela que controla a entrada dos utilizadores no parque [Rea24].
- ***Google Firebase***: Armazena os dados recolhidos pelos ESP32 e facilita o acesso em tempo real [Goo24a].

2.4.2 Funcionamento Geral

- **Monitorização da Ocupação**: Os ESP32 recolhem dados sobre a ocupação de cada lugar de estacionamento [Esp24a].
- **Envio de Dados**: As informações de ocupação são enviadas diretamente para uma base de dados no *Google Firebase* [Goo24a].
- **Acesso aos Dados**: Utilizadores podem acessar os dados em tempo real através de uma aplicação móvel para verificar a disponibilidade de lugares.
- **Controlo de Acesso**: A entrada no parque é controlada por uma cancela operada por um motor servo, com base nas informações de ocupação.

Capítulo 3

Análise do Sistema

Com o objetivo de criar uma solução para melhorar a busca por vagas de estacionamento em áreas urbanas densas, é necessária uma análise que inclua uma descrição dos casos de uso, especificando as interações dos utilizadores com o sistema, requisitos funcionais e não funcionais.

3.1 Casos de Uso

Casos de uso descrevem interações entre utilizadores e sistema, detalhando passos para alcançar objetivos específicos, focando funcionalidades e requisitos do projeto [Gee24b].

3.1.1 Procurar Vagas de Estacionamento

- **Ator**: Utilizador
- **Pré-condições**: Utilizador está autenticado no sistema através da interface.
- **Fluxo Principal**:
  1. O utilizador abre a interface do sistema.
  2. O utilizador procura a cidade que pretende na secção de pesquisa.
  3. O sistema exibe um mapa com os parques disponíveis na cidade pesquisada.
  4. O utilizador seleciona, no mapa, o parque que pretende visualizar.
  5. O sistema exibe um *PopUp* [Get24] com estado dos lugares, em tempo real.
  6. O utilizador visualiza os lugares disponíveis e escolhe um.
  7. O utilizador, se pretender, seleciona a opção para mostrar o caminho até ao parque.

3.1.2 Reservar Lugar para Necessidades Especiais

- **Ator**: Utilizador com necessidades especiais
- **Pré-condições**: Utilizador está autenticado e registado com necessidades especiais.
- **Fluxo Principal**:
  1. O utilizador abre a interface do sistema.
  2. O utilizador procura a cidade que pretende na secção de pesquisa.
  3. O sistema exibe um mapa com os parques disponíveis, com lugares para necessidades especiais, na cidade pesquisada.
  4. O utilizador seleciona, no mapa, o parque que pretende visualizar.
  5. O sistema exibe um *PopUp* [Get24] com estado dos lugares, em tempo real, juntamente com a possibilidade de reserva.
  6. O utilizador visualiza os lugares disponíveis, escolhe um e, se pretender, reserva.
- **Pós-condições**: O lugar para necessidades especiais fica reservado para o utilizador.

3.1.3 Abrir Cancela do Lugar Reservado

- **Ator**: Utilizador com necessidades especiais
- **Pré-condições**: Utilizador está autenticado, o lugar está reservado e a hora atual está entre os tempos definidos na reserva.
- **Fluxo Principal**:
  1. O utilizador chega ao lugar reservado na hora da reserva.
  2. O utilizador abre a interface do sistema.
  3. O utilizador abre, na interface do sistema, a secção **As minhas reservas**.
  4. O utilizador seleciona a opção **Abrir Cancela** na reserva.
  5. O sistema abre a cancela.
- **Pós-condições**: A cancela do lugar reservado está aberta.

3.1.4 Analisar a plataforma

- **Ator**: Utilizador com permissões de administrador
- **Pré-condições**: Utilizador está autenticado e tem permissões de administrador.
- **Fluxo Principal**:
  1. O utilizador abre a interface do sistema.
  2. O utilizador abre, na interface do sistema, uma das secções **Arquivo** ou ***Dashboard*** [Tab24].
  3. O sistema disponibiliza os dados referentes à secção selecionada.
  4. O utilizador analisa os dados.

3.2 Requisitos Funcionais

Requisitos funcionais descrevem o que o sistema deve fazer, suas funcionalidades e comportamentos específicos [Gee24c].

3.2.1 Autenticação e Registo

- O sistema deve permitir que os utilizadores se registem e autentiquem na plataforma.
- O sistema deve permitir que utilizadores registem suas necessidades especiais.

3.2.2 Pesquisa e Reserva de Lugares

- O sistema deve exibir num mapa com a disponibilidade de vagas em tempo real.
- O sistema deve permitir que os utilizadores reservem vagas adaptadas para necessidades especiais.

3.2.3 Atualização em Tempo Real

- O sistema deve atualizar a disponibilidade das vagas em tempo real.

3.2.4 Controle de Acesso

- O sistema deve permitir que utilizadores com necessidades especiais abram a cancela do lugar reservado através da interface.

3.3 Requisitos Não Funcionais

Requisitos não funcionais definem como o sistema deve operar, incluindo desempenho, segurança e usabilidade, entre outros [Gee24c].

3.3.1 Desempenho

- O sistema deve responder às solicitações do utilizador em menos de 2 segundos.
- O sistema deve processar a atualização de disponibilidade de vagas em tempo real com um atraso entre 10 e 30 segundos.

3.3.2 Segurança

- O sistema deve garantir que os dados dos utilizadores estejam protegidos contra acesso não autorizado.
- As transações de reserva e abertura de cancela devem ser seguras e encriptadas.

3.3.3 Usabilidade e Compatibilidade

- A interface do sistema deve ser intuitiva e fácil de usar, permitindo que utilizadores de todas as idades naveguem facilmente.
- O sistema deve estar disponível em múltiplas plataformas (***iOS***, ***Android***, ***Web***).

3.3.4 Escalabilidade

- O sistema deve ser capaz de suportar um grande número de utilizadores simultâneos sem degradação de desempenho.

3.3.5 Confiabilidade

- O sistema deve ter mecanismos de recuperação em caso de falhas.

Capítulo 4

Arquiteturas do Sistema

O sistema ***ParkLookUp*** possui áreas interna e externa para veículos. No parque interno, sensores detetam se os lugares estão ocupadas, enquanto no parque externo, câmaras realizam essa verificação. O sistema garante uma gestão eficiente e em tempo real dos lugares disponíveis, facilitando o estacionamento para os utilizadores.

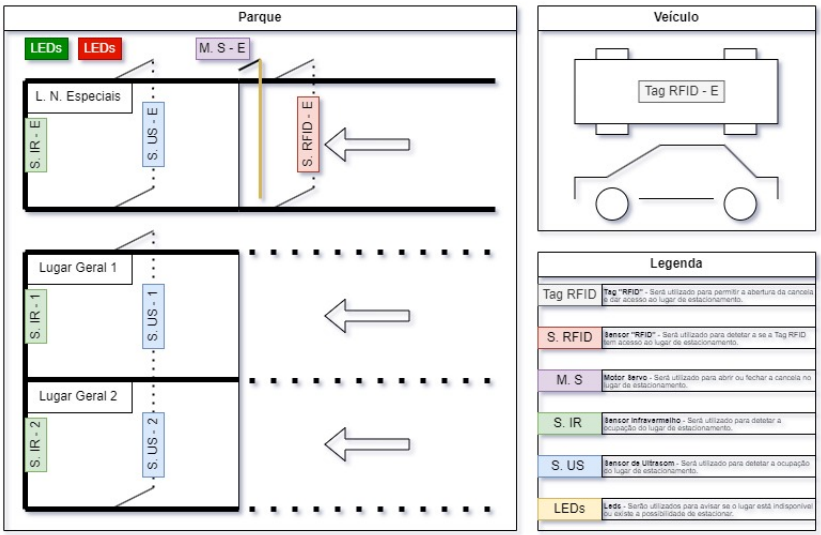

Figura 4.1: Desenho do Parque de Estacionamento Interno

O parque interior possui 3 lugares (lugares **1**, **2** e **3**), um deles equipado com uma cancela controlada por um **motor servo** (*SG-90* [Tow24]) para necessidades especiais. Todas os lugares utilizam sensores **infravermelhos** (*MH-B* [Ele24]) e **ultrassónicos** (*HC-SR04* [Dig24]) para detetar a ocupação. A cancela pode ser aberta com *tag* ***RFID*** [Tec24] (*MFRC522* [Han24]), ou pelo sistema de interface *web*, garantindo acessibilidade e segurança.

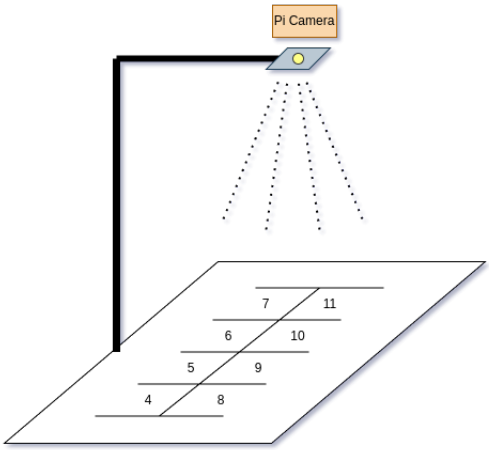

Figura 4.2: Desenho do Parque de Estacionamento Externo

O parque exterior possui 8 lugares (lugares **4** a **11**), todos monitorizados por uma câmara (*Pi Camera* [Ras24b]). As imagens são pós-processadas através de um modelo de inteligência artificial que permite detetar a ocupação individual de cada lugar.

4.1 Base de Dados

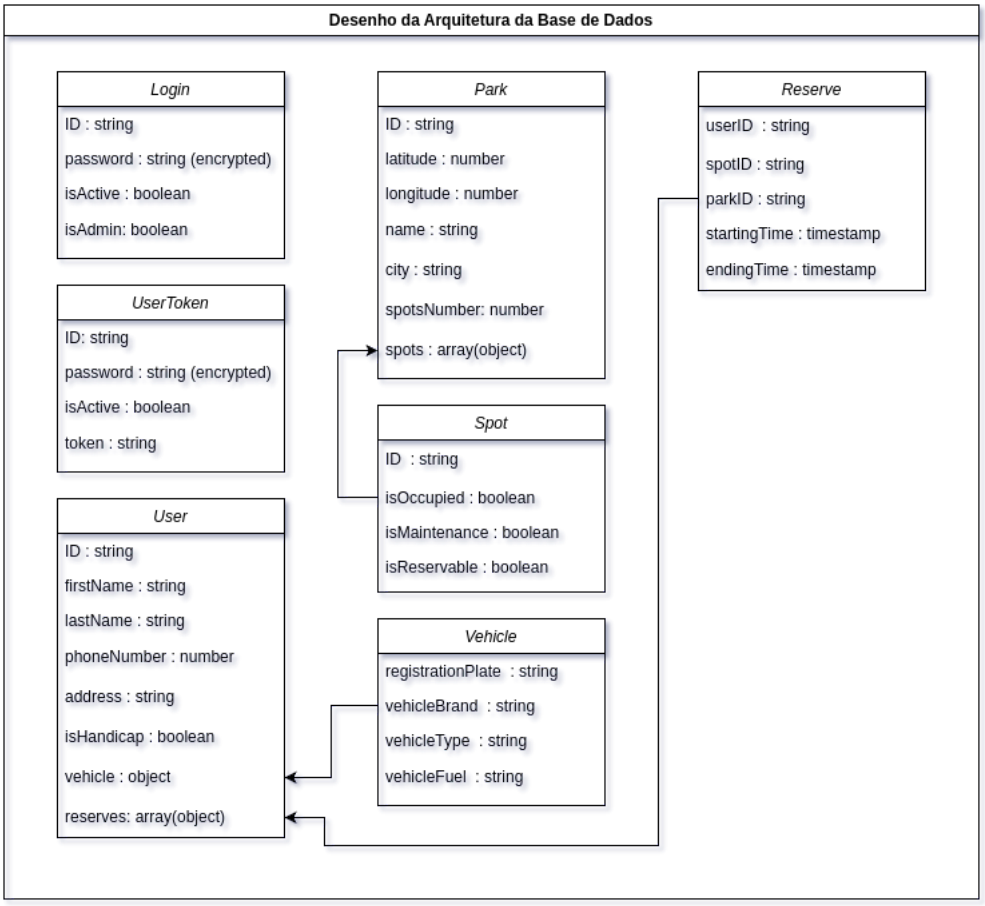

Figura 4.3: Desenho da Arquitetura da Base de Dados

A arquitetura da base de dados para o sistema de estacionamento inteligente ***ParkLookUp*** é composta por vários blocos de dados interligados, cada um desempenhando um papel específico na gestão das informações de estacionamento. Aqui está uma descrição simples das blocos de dados e os seus campos:

4.1.1 Login

Armazena as credenciais de acesso dos utilizadores, permitindo a autenticação segura no sistema.

- **ID**: Identificador único (string)
- **password**: Senha do utilizador (string criptografada)
- **isActive**: Indica se a conta está ativa (boolean)

- **isAdmin**: Indica se o utilizador é administrador (boolean)

4.1.2 UserToken

Gere *tokens* de autenticação para sessões de utilizadores, garantindo acesso seguro e autorizado.

- **ID**: Identificador único (string)
- **password**: Senha do utilizador (string criptografada)
- **isActive**: Indica se a conta está ativa (boolean)
- **token**: Chave de autenticação (string)

4.1.3 User

Contém informações pessoais dos utilizadores, como nome, contacto e detalhes sobre necessidades especiais.

- **ID**: Identificador único (string)
- **firstName**: Primeiro nome do utilizador (string)
- **lastName**: Sobrenome do utilizador (string)
- **phoneNumber**: Número de telefone do utilizador (number)
- **address**: Endereço do utilizador (string)
- **isHandicap**: Indica se o utilizador possui necessidades especiais (boolean)
- **vehicle**: Lista de veículos associados ao utilizador (object)
- **reserves**: Lista de reservas associadas ao utilizador (array(object))

4.1.4 Park

Regista detalhes dos parques de estacionamento, incluindo localização, nome, cidade e número de vagas disponíveis.

- **ID**: Identificador único (string)
- **latitude**: Latitude do parque de estacionamento (number)
- **longitude**: Longitude do parque de estacionamento (number)
- **name**: Nome do parque de estacionamento (string)

- **city**: Cidade onde o parque está localizado (string)
- **spotsNumber**: Número total de vagas no parque (number)
- **spots**: Array de objetos de lugares de estacionamento (array(objetos))

4.1.5 Spot

Monitora o estado individual das vagas de estacionamento, indicando se estão ocupadas, em manutenção ou disponíveis para reserva.

- **ID**: Identificador único (string)
- **isOccupied**: Indica se a vaga está ocupada (boolean)
- **isMaintenance**: Indica se a vaga está em manutenção (boolean)
- **isReservable**: Indica se a vaga pode ser reservada (boolean)

4.1.6 Vehicle

Armazena informações sobre os veículos dos utilizadores, como placa de registo, marca, tipo e combustível.

- **registrationPlate**: Placa de registro do veículo (string)
- **vehicleBrand**: Marca do veículo (string)
- **vehicleType**: Tipo de veículo (string)
- **vehicleFuel**: Tipo de combustível do veículo (string)

4.1.7 Reserve

Regista as reservas de lugares de estacionamento, incluindo dados do utilizador, lugar reservada, parque e horários de início e término.

- **userID**: Identificador único do utilizador que fez a reserva (string)
- **spotID**: Identificador único da vaga reservada (string)
- **parkID**: Identificador único do parque de estacionamento (string)
- **startingTime**: Hora de início da reserva (timestamp)
- **endingTime**: Hora de término da reserva (timestamp)

Essa estrutura permite a gestão eficiente das informações de utilizadores, veículos, vagas de estacionamento e reservas, garantindo que o sistema funcione de maneira organizada e eficaz. A tipagem de variáveis seguiu o padrão da tecnologia ***MongoDB*** [Mon24a].

4.2 Desenho da Arquitetura do Sistema

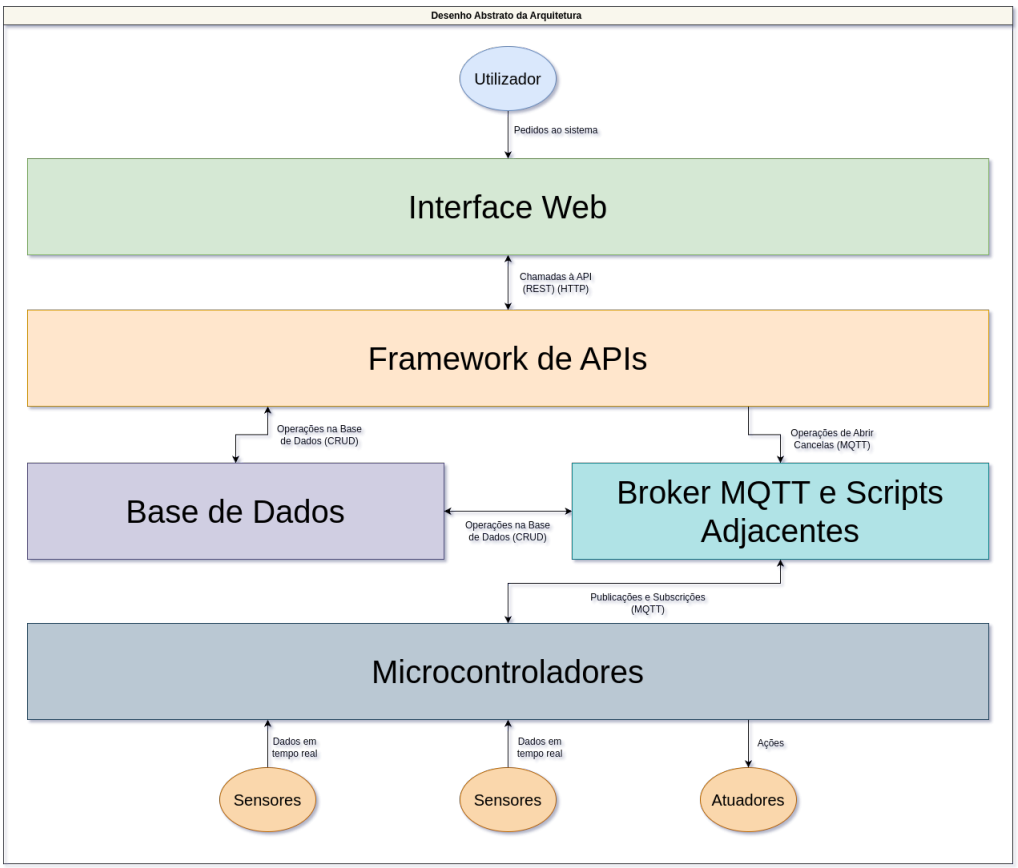

Figura 4.4: Desenho da Arquitetura - Abstrato

O desenho da arquitetura do sistema, em formato abstrato, do sistema de ***ParkLookUp*** é composto por várias camadas interconectadas, cada uma desempenhando um papel essencial no funcionamento do sistema. Aqui está uma descrição simples de cada camada:

4.2.1 Utilizador

- Interage com o sistema através da Interface *Web* para acessar informações e realizar operações.

4.2.2 Interface *Web*

- Proporciona uma interface gráfica para os utilizadores, facilitando a interação com o sistema através de navegadores web.

4.2.3 *Framework de APIs*

- Garante a comunicação entre a Interface Web e outros componentes do sistema, permitindo o acesso a dados e serviços [AWS23]. Esta camada recorrer a operações *REST* [Red24] cuja a comunicação é feita através de *HTTP* [Clo24a].

4.2.4 Base de Dados

- Armazena todas as informações necessárias, como dados de utilizadores, veículos, reservas e estado das vagas de estacionamento.

4.2.5 *Broker MQTT* e *Scripts* Adjacentes

- Facilita a comunicação eficiente entre os micro-controladores [Gee24d] e o sistema central, utilizando o protocolo *MQTT* [Emq24b] para a transmissão de dados.

4.2.6 Micro-controladores

- Controlam os sensores no campo, coletando dados sobre a ocupação das vagas de estacionamento [Gee24d; Zip23].

4.2.7 Sensores e Atuadores

- Monitorizam os lugares de estacionamento, detetando a presença ou ausência de veículos e enviando esses dados para os micro-controladores [Gee24d; Zip23]. Também executam operações de abertura e fecho de cancelas.

Esta estrutura modular permite uma gestão eficiente do sistema de estacionamento, garantindo uma comunicação fluida entre utilizadores, dispositivos de campo e a central de dados.

Capítulo 5

Implementação do Sistema

Após a análise detalhada da arquitetura descrita no capítulo anterior, procede-se à implementação da mesma, utilizando tecnologias específicas para cada tarefa. Essa fase abrange a escolha e aplicação das tecnologias adequadas para garantir a funcionalidade e eficiência do sistema proposto.

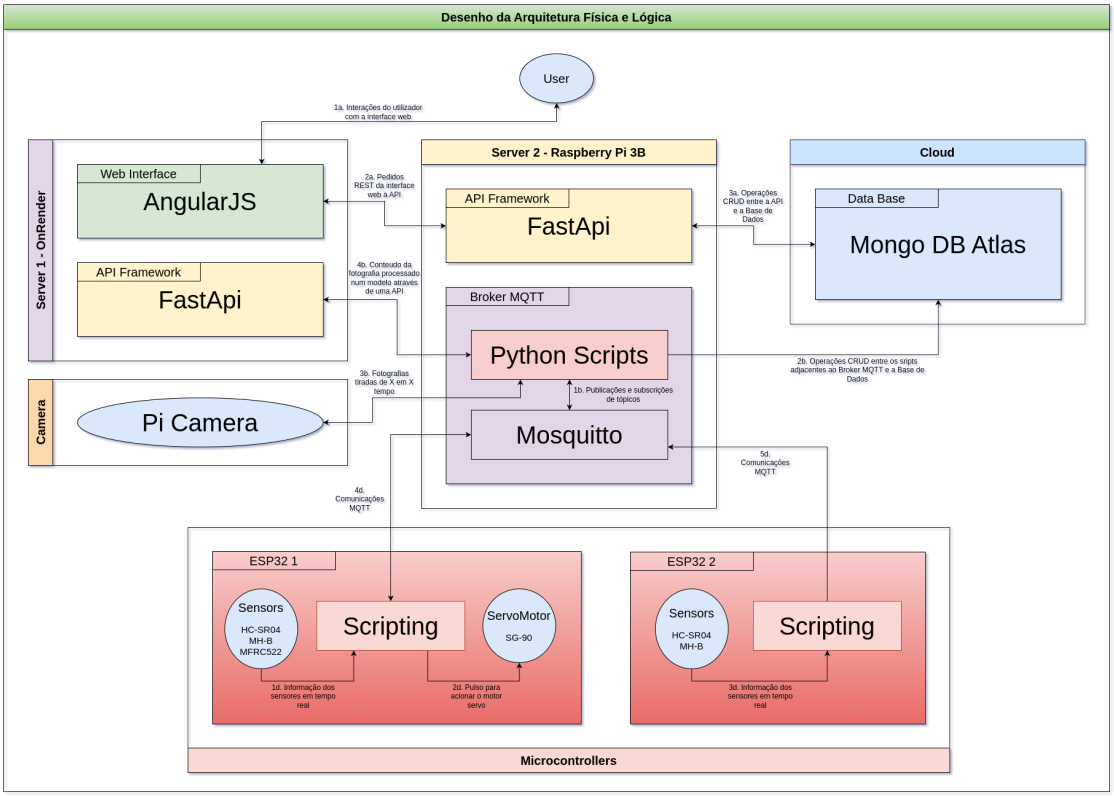

Figura 5.1: Desenho da Arquitetura com Recurso às Tecnologias

5.1 Lista da tecnologias utilizadas

5.1.1 Hardware

- **Raspberry Pi 3 Model B** - Computador de placa única [Bae24], centralizar a maior parte dos dados neste sistema [Ras24a].
- **Pi Camera** - Câmara para *Raspberry Pi*, utilizada para capturar imagens do parque de estacionamento exterior [Ras24b].
- **ESP32** - Micro-controlador com que executa o tratamento dos dados recolhidos pelos sensores ou a execução de atuadores [Esp24a].
- **HC-SR04** - Sensor ultrassónico que mede distâncias usando ondas sonoras. Deteta a presença de um veículo no parque de estacionamento interior [Dig24].
- **MH-B** - Sensor de presença que deteta movimento. Deteta a presença de um veículo no parque de estacionamento interior [Ele24].
- **MFRC522** - Leitor de *RFID* para leitura de *tags RFID* [Tec24]. Usado para informar se os lugares reserváveis entraram em manutenção.
- **SG-90** - Motor servo [Rea24]. Utilizado para abir e fechar as cancelas dos lugares reserváveis [Tow24].

5.1.2 Software e Bibliotecas

- **AngularJS**: *Framework front-end* que permite criar uma interface interativa e dinâmica para visualização em tempo real de vagas de estacionamento disponíveis [Ang24].
- **FastAPI**: *Framework back-end* de alta performance para construir *APIs RESTful* rápidas, usando protocolo *HTTP*, gerindo dados de disponibilidade e reservas de estacionamento [AWS23; Clo24a; Red24; Fas24].
- **Mosquitto**: *Broker MQTT* que facilita a comunicação em tempo real entre os micro-controladores e o servidor, garantindo atualizações rápidas e precisas [Emq24a; Emq24b; Mos24].
- **MongoDB Atlas**: Serviço de base de dados em *cloud* escalável, armazenando informações de utilizadores, reservas e disponibilidade de lugares de estacionamento [Mon24b].
- **Roboflow**: Plataforma computacional que processa as imagens da câmara, detetando a ocupação dos lugares estacionamento [Rob24].

5.2 Funcionamento do Sistema

Para descrever o funcionamento do sistema ***ParkLookUp*** deve ser realizada uma análise das conexões entre seus diferentes blocos funcionais. Cada bloco representa um componente específico do sistema, e as conexões entre eles, identificadas por números seguidos de letras (por exemplo, 1A, 2B), indicam a interação e o fluxo de informações entre esses componentes. No seguimento, exploraremos detalhadamente cada uma dessas conexões, explicando seu papel específico e como contribuem para a eficácia geral do sistema.

5.2.1 1a. Interação Utilizador - Interface *Web*

- Esta interação permite ao utilizador registar uma conta, autenticar, procurar parques de estacionamento, reservar uma vaga e abrir cancelas. Este processo centraliza todas as ações necessárias para a gestão de estacionamento, garantindo uma experiência integrada e conveniente para os utilizadores.
- O uso de *AngularJS* [Ang24] deveu-se:
  - **Desempenho Dinâmico**: Atualização dinâmica da interface do utilizador.
  - **Estrutura Modular**: Facilita a manutenção.
  - ***Binding* Bidirecional**: Sincronização automática de dados.
  - **Comunidade Ativa**: Suporte contínuo e muitos recursos.
  - **Conhecimento prévio**: Já havia conhecimento sobre a tecnologia em questão, permitindo mais agilidade no desenvolvimento do sistema.
- Todo o código desenvolvido em ***AngularJS*** [Ang24] encontra-se na plataforma ***Github*** [Git24a].

5.2.2 2a. Interação Interface *Web* - *API Framework*

- Esta interação envolve a interface web enviando pedidos à API para obter listagens de parques e lugares disponíveis, verificar se o utilizador está ativo, autenticar utilizadores, realizar operações como abrir cancelas, entre outras. A API processa esses pedidos e retorna as respostas necessárias, facilitando a gestão eficiente do parque.
- O uso de *FastAPI* [Fas24] deveu-se:
  - **Desempenho Rápido**: *FastAPI* oferece alta performance.
  - **Validação Automática**: Simplifica a validação de dados.
  - **Documentação Automática**: Gera documentação interativa.
  - **Segurança**: Todas as comunicações são seguras, utilizando *Bearer Token* [Swa24] para habilitar as comunicações.

  - **Fácil Implementação**: Agilidade em implementar *Endpoints* [Gee24a] para o sistema.

- Todo o código desenvolvido em ***FastApi*** [Fas24], para esta interação, encontra-se na plataforma ***Github*** [Git24d].

5.2.3 3a. Interação *API Framework* - Base de Dados

- A interação entre a *API Framework* [Fas24] e a base de dados na *cloud* [Mic24] envolve a transmissão de dados para registar contas, aceder a dados para autenticação, e consultar informações sobre parques e lugares, entre outros. A *API* [AWS23] envia consultas à base de dados, que retorna as informações necessárias para as operações solicitadas.

- O uso de *MongoDB Atlas* [Mon24b] deveu-se:

  - **Escalabilidade**: *MongoDB Atlas* [Mon24b] escala automaticamente conforme necessário.
  - **Gestão Simplificada**: Reduz a complexidade da administração de bases de dados.
  - **Segurança**: Oferece segurança robusta com backups automáticos.
  - **Conhecimento prévio**: Já havia conhecimento sobre a tecnologia em questão, permitindo mais agilidade no desenvolvimento do sistema.

- Todo o código desenvolvido para trabalhar com ***MongoDB Atlas*** [Mon24b], para esta interação, encontra-se na plataforma ***Github*** [Git24d; Git24f].

5.2.4 1b. Interação *Python Scripts* - *Broker MQTT*

- A interação entre *Python Scripts* [Pyt24b] e um *Broker MQTT* [Emq24a], no *Raspberry Pi* [Ras24a], envolve os *scripts* recebendo e publicando mensagens via protocolo *MQTT* [Emq24b] para o *broker Mosquitto* [Emq24a; Mos24]. Esta comunicação permite a troca de dados em tempo real entre dispositivos conectados, facilitando a gestão e monitorização de operações no sistema.

- O uso de *Python*, com recurso à biblioteca *Paho-MQTT*, [Pyt24b; Pyt24a] deveu-se:

  - **Facilidade de Uso**: *Python* [Pyt24b] é intuitivo e fácil de aprender.
  - **Biblioteca *Paho-MQTT***: Suporte robusto e eficiente para *MQTT* [Emq24b].
  - **Integração Simples**: Integra-se facilmente com outras tecnologias e serviços.

- Todo o código desenvolvido para trabalhar com ***Python*** [Pyt24b], para esta interação, encontra-se na plataforma ***Github*** [Git24f].

5.2.5 2b. Interação *Python Scripts* - Base de Dados

- Os *Python Scripts* [Pyt24b], que subscrevem e publicam mensagens via *broker MQTT* [Emq24a; Emq24b], também interagem com o *MongoDB Atlas* [Mon24b] para operações *CRUD* [Mon23]. Eles processam as mensagens *MQTT* [Emq24b], extraem os dados relevantes e realizam consultas para criar, ler, atualizar ou apagar registos na base de dados, integrando dados em tempo real.

- O uso de *Python*, com recurso à biblioteca *PyMongo*, [Pyt24b; PyM24] deveu-se:

  - **Compatibilidade**: *Paho-MQTT* [Pyt24a] funciona bem com *MongoDB Atlas* [Mon24b].
  - **Eficiência**: Facilita a comunicação em tempo real.
  - **Integração**: Permite fácil integração de dados entre dispositivos *IoT* [IBM24] e *MongoDB* [Mon24b].

- Todo o código desenvolvido para trabalhar com ***Python*** [Pyt24b], para esta interação, encontra-se na plataforma ***Github*** [Git24f].

5.2.6 3b. Interação *Pi Camera* - *Python Scripts*

- Os *Python Scripts* [Pyt24b] interagem com a *Pi Camera* [Ras24b], no *Raspberry Pi* [Ras24a], para capturar imagens. Esses *scripts* configuram a câmara, ajustam parâmetros e tiram fotos com comandos simples. As imagens serão usadas para detetar se os lugares estão ocupados ou não, facilitando a monitorização no parque de estacionamento.

- O uso de *Pi Camera* [Ras24b] deveu-se:

  - **Integração Simples**: *Pi Camera* [Ras24b] é ideal para *Raspberry Pi* [Ras24a].
  - **Qualidade Superior**: Oferece a resolução suficiente para a deteção precisa da ocupação dos lugares no parque de estacionamento.

- Todo o código desenvolvido para trabalhar com *Pi Camera* [Pyt24b], para esta interação, encontra-se na plataforma ***Github*** [Git24f].

5.2.7 4b. Interação *API Framework* - *Python Scripts*

- Os *Python Scripts* [Pyt24b] enviam uma solicitação através da *API FastAPI* [Fas24; AWS23] para processar uma imagem, utilizando inteligência artificial com a tecnologia *RoboFlow* [Rob24]. Esta biblioteca oferece modelos pré-treinados para análise de imagens, permitindo identificar a ocupação dos lugares no parque de estacionamento de maneira eficiente e precisa.

- O uso de *Roboflow* [Rob24], e a sua utilização num servidor diferente, deveu-se:
  - **Modelos Pré-Treinados**: *Roboflow* [Rob24] oferece modelos de inteligência artificial prontos para uso, economizando tempo de desenvolvimento.
  - **Escalabilidade**: Hospedar *Roboflow* [Rob24] em servidor dedicado melhora o desempenho e a disponibilidade dos modelos.
  - **Falta da Capacidade**: O *Raspberry Pi Model 3* [Ras24a], com processador de 32 bits, não suporta todas as bibliotecas avançadas necessárias para o sistema *RoboFlow* [Rob24].
- Todo o código desenvolvido para trabalhar com *RoboFlow* [Rob24], para esta interação, encontra-se na plataforma ***Github*** [Git24e; Git24f].

5.2.8 1d. & 2d. Interação Sensores - *Scripting* - Motor Servo

- O *ESP32* [Esp24a] interage com sensores infravermelhos [Ele24], ultrassónicos [Dig24] e *RFID* [Tec24; Han24] para monitorizar proximidade e identificação. Além disso, controla um *servomotor* [Rea24] para operar cancelas. Mantém-se constantemente ligado para receber comunicações em tempo real via *MQTT* [Emq24b], garantindo operações eficientes e responsivas no sistema.
- O uso de *ESP32* [Esp24a], deveu-se:
  - **Resposta Imediata**: Mantendo o *ESP32* [Esp24a] sempre ligado, garante-se que responda rapidamente às comunicações em tempo real, crucial para operações eficientes no parque de estacionamento.
  - **Atualizações Contínuas**: Permite receber e processar atualizações instantâneas, assegurando que o sistema esteja sempre sincronizado e operacional.
- Todo o código desenvolvido para trabalhar com o *ESP32* [Esp24a], para esta interação, encontra-se na plataforma ***Github*** [Git24b].

5.2.9 3d. Interação Sensores - *Scripting*

- Os sensores infravermelhos [Ele24], ultrassónicos [Dig24] conectados ao *ESP32* [Esp24a] fornecem dados de proximidade. O *ESP32* [Esp24a] entra em *Sleep Mode* [Esp24b] para economizar de energia e periodicamente envia dados para um *broker MQTT* [Emq24a; Emq24b], assegurando atualizações regulares e eficiência energética no sistema.
- O uso de *Sleep Mode* [Esp24b], deveu-se:
  - **Economia de Energia**: Reduz o consumo durante períodos, em rotina, para prolongar a vida útil da bateria.

  - **Menor Impacto Ambiental**: Minimiza a pegada de carbono ao otimizar o uso de energia no sistema.

- Todo o código desenvolvido para trabalhar com o *ESP32* [Esp24a], para esta interação, encontra-se na plataforma ***Github*** [Git24c].

5.2.10 4d. & 5d. Interação *Scripting - Broker MQTT*

- Os *scripts* programados para as placas *ESP32* [Esp24a] interagem com o *broker Mosquitto* [Emq24a; Mos24] usando a biblioteca *PubSubClient* [Ard24] para comunicação via *MQTT* [Emq24b]. Esta integração permite o envio e recebimento de dados em tempo real, essencial para operações eficientes e atualizações contínuas no sistema.

- O uso de *PubSubClient* [Ard24], deveu-se:

  - **Simplicidade de Implementação**: Facilita a comunicação *MQTT* [Mos24] entre *ESP32* [Esp24a] e *Mosquitto* [Emq24a; Mos24] com código claro e conciso.
  - **Robustez e Confiabilidade**: Oferece recursos robustos para lidar com mensagens *MQTT* [Emq24b] de forma confiável no sistema.

- Todo o código desenvolvido para trabalhar com o *ESP32* [Esp24a], para esta interação, encontra-se na plataforma ***Github*** [Git24b; Git24c].

5.3 SSH & Wireguard

O uso de ***SSH*** [Gee24e] foi fundamental para interagir remotamente com o *Raspberry Pi* [Ras24a], que servia como núcleo central do sistema do ***ParkLookup***, centralizando e distribuindo dados conforme necessário. O acesso ***SSH*** [Gee24e] permitiu gerir o *Raspberry Pi* [Ras24a] de forma segura e eficiente, facilitando operações de manutenção e configuração à distância. Este acesso foi configurado através do *router* [Clo24b] de rede, garantindo que o *Raspberry Pi* [Ras24a] estivesse acessível externamente apenas com as devidas medidas de segurança.

Além disso, foi implementada uma ***VPN*** utilizando ***WireGuard*** [Wir24] para estabelecer comunicações seguras e diretas entre um computador pessoal e o *Raspberry Pi* [Ras24a]. Embora o ***WireGuard*** [Wir24] ofereça uma conexão *peer-to-peer* [gee24] eficiente, limitou as comunicações a *IPs* [Kas24] específicos. Isso proporcionou uma camada adicional de segurança e privacidade para as interações entre dispositivos dentro da rede de dispositivos controlados, garantindo que os dados sensíveis fossem transmitidos de maneira segura e protegida contra acessos não autorizados.

Capítulo 6

Testes ao Sistema

Devido às limitações de transporte causadas pela maqueta do sistema **ParkLookUp**, os testes foram difíceis de executar. No entanto, foi realizado um teste com um utilizador alheio ao desenvolvimento do projeto. Este teste seguiu a Técnica *Cognitive Walkthrough* [Int24] que se baseia em alguns pontos como:

- **Exploração Mental Empática**: Colocando-se nos sapatos do utilizador.
- **Procura ativa por obstáculos**: Descobrindo barreiras à usabilidade.
- **Padrões de Avaliação Perspicazes**: Critérios de análise afiados para descobrir falhas.
- **Processo Reflexivo e Iterativo**: Aprimorando continuamente a compreensão da usabilidade.

O teste gerou um vídeo onde se pode ver a interação do utilizador com o sistema tanto na interface *Web* como com a maqueta real do parque de estacionamento. O vídeo seguirá em anexo a este relatório.

Capítulo 7

Conclusões

O sistema **ParkLookUp** visa solucionar os desafios decorrentes da urbanização crescente e do aumento da densidade populacional, que causam um tráfego urbano intenso e tornam difícil encontrar lugares de estacionamento disponíveis. Ao facilitar a busca por vagas em estacionamentos através de mapas e destacar a disponibilidade em tempo real, o sistema **ParkLookUp** busca aprimorar a eficiência e a experiência dos utilizadores. Adicionalmente, o sistema atende às necessidades de pessoas com necessidades especiais, permitindo que se registem na plataforma e reservem vagas adaptadas às suas condições específicas.

Para adquirir um conhecimento profundo sobre o tema, foram analisados quatro sistemas semelhantes com o objetivo de preparar o sistema **ParkLookUp** da melhor forma possível para os utilizadores. Esta análise focou tanto na interface do utilizador quanto na implementação física em um estacionamento real.

O sistema pretendia atender aos requisitos funcionais, não funcionais e ainda aos casos de usos propostos durante a fase de análise, pois, em termos gerais, são os requisitos mínimos a contemplar por um sistema *IoT* que pretenda interagir com utilizadores. No entanto, os mesmos não foram aprofundados integralmente devido à carência de tempo para realizar o projeto.

Durante a análise do sistema, foram abordados aspetos cruciais como o desenho da interface gráfica, a arquitetura da base de dados e a arquitetura física. Estes elementos visam garantir uma interação intuitiva, uma gestão eficiente dos dados e uma implementação prática e robusta do sistema em ambientes reais.

A implementação do sistema destaca-se pela adoção dos protocolos ***MQTT*** e ***HTTP*** para comunicação eficiente e segura. Utiliza-se **inteligência artificial** para detetar a ocupação dos lugares através de imagens, melhorando a gestão de estacionamento. O ***SSH*** permite interação remota segura com o ***Raspberry Pi***, garantindo administração flexível e acessível do sistema **ParkLookUp**.

Em suma, o projeto, segue as ideias e planeamentos propostos, no entanto faltaram alguns pontos a melhorar:

- **A interface** - Sendo um dos pontos de contacto com o utilizador, deveria ter sido mais trabalhada, apesar de fazer todos os principais propósitos do projeto.
- **A segurança** - Como um dos pontos fundamentais de um sistema com interface *web*, garantir que o sistema respeita o *Top 10 OWASP* [OWA24], seria um ponto a melhorar.
- **Os testes** - Seria a garantia que os utilizadores não encontrariam comportamentos estranhos no sistema, assim, uma bateria de testes mais exaustiva seria necessária.
- **Eficiência Energética** - Eficiência energética é crucial para minimizar o consumo de energia. Seria necessário rever melhor este ponto.

Bibliografia

[Ang24] AngularJs. Consultado em 2024/02/20. 2024. URL: https://angularjs.org/ (citado nas páginas 28, 29).

[Ard24] Arduino. *PubSubClient*. Consultado em 2023/12/04. 2024. URL: https://www.arduino.cc/reference/en/libraries/pubsubclient/ (citado na página 33).

[AWS23] AWS. *API*. Consultado em 2023/11/17. 2023. URL: https://aws.amazon.com/pt/what-is/api/ (citado nas páginas 25, 28, 30, 31).

[Bae24] BaeSystems. *SBC*. Consultado em 2024/02/20. 2024. URL: https://www.baesystems.com/en-us/definition/what-are-single-board-computers (citado na página 28).

[C S20] Aswathy C S. «Low Cost Smart Parking System with IOT». Em: *International Journal for Research in Applied Science and Engineering Technology* 8 (fev. de 2020), pp. 273–278. DOI: 10.22214/ijraset.2020.2040 (citado nas páginas 3, 8).

[Clo24a] Cloudflare. Consultado em 2024/02/20. 2024. URL: https://www.cloudflare.com/learning/ddos/glossary/hypertext-transfer-protocol-http/ (citado nas páginas 25, 28).

[Clo24b] Cloudflare. *Router*. Consultado em 2024/02/20. 2024. URL: https://www.cloudflare.com/learning/network-layer/what-is-a-router/ (citado na página 33).

[Dig24] Digikey. *HC-SR04*. Consultado em 2024/02/20. 2024. URL: https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/692/3942_Web.pdf (citado nas páginas 8, 19, 28, 32).

[Ele24] Electroduino. *MH-B*. Consultado em 2024/02/20. 2024. URL: https://www.electroduino.com/what-is-ir-sensor-module-how-ir-sensor-module-works/ (citado nas páginas 19, 28, 32).

[Emq24a] Emqx. *Broker*. Consultado em 2024/02/20. 2024. URL: https://www.emqx.com/en/blog/the-ultimate-guide-to-mqtt-broker-comparison (citado nas páginas 28, 30–33).

[Emq24b] Emqx. *MQTT*. Consultado em 2024/02/20. 2024. URL: https://www.emqx.com/en/blog/the-easiest-guide-to-getting-started-with-mqtt (citado nas páginas 25, 28, 30–33).

[Esp24a] Espressif. *ESP32*. Consultado em 2024/02/20. 2024. URL: https://www.espressif.com/en/products/socs/esp32 (citado nas páginas 10, 11, 28, 32, 33).

[Esp24b] Espressif. *Timer SleepModes*. Consultado em 2024/02/20. 2024. URL: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/sleep_modes.html (citado na página 32).

[Fas24] FastApi. Consultado em 2024/02/20. 2024. URL: https://fastapi.tiangolo.com/ (citado nas páginas 28–31).

[Gee24a] GeekForGeeks. *EndPoint*. Consultado em 2024/02/20. 2024. URL: https://www.geeksforgeeks.org/what-is-an-endpoint-in-security/ (citado na página 30).

[Gee24b] Geeksforgeeks. Consultado em 2024/02/20. 2024. URL: https://www.geeksforgeeks.org/designing-use-cases-for-a-project/ (citado na página 13).

[Gee24c] Geeksforgeeks. Consultado em 2024/02/20. 2024. URL: https://www.geeksforgeeks.org/functional-vs-non-functional-requirements/ (citado nas páginas 15, 16).

[Gee24d] Geeksforgeeks. Consultado em 2024/02/20. 2024. URL: https://www.geeksforgeeks.org/microcontroller-and-its-types/ (citado na página 25).

[Gee24e] Geeksforgeeks. *SSH*. Consultado em 2023/12/04. 2024. URL: https://www.geeksforgeeks.org/introduction-to-sshsecure-shell-keys/ (citado na página 33).

[gee24] geeksforgeeks. *PeerToPeer*. Consultado em 2024/02/20. 2024. URL: https://www.geeksforgeeks.org/what-is-p2p-peer-to-peer-process/ (citado na página 33).

[Get24] Getresponse. Consultado em 2024/02/20. 2024. URL: https://www.getresponse.com/blog/what-are-popups (citado nas páginas 13, 14).

[Git24a] Github. *GithubAngular*. Consultado em 2024/02/20. 2024. URL: https://github.com/VascoGomesWork/Projecto-IS/tree/main/Code/OnRender/AngularJs (citado na página 29).

[Git24b] Github. *GithubESP32*$_1$. Consultado em 2024/02/20. 2024. URL: https://github.com/VascoGomesWork/Projecto-IS/tree/main/Code/ESP32-1 (citado nas páginas 32, 33).

[Git24c] Github. *GithubESP32*$_2$. Consultado em 2024/02/20. 2024. URL: https://github.com/VascoGomesWork/Projecto-IS/tree/main/Code/ESP32-2 (citado na página 33).

[Git24d] Github. *GithubFastApi1*. Consultado em 2024/02/20. 2024. URL: https://github.com/VascoGomesWork/Projecto-IS/tree/main/Code/RPi/code/fastApi (citado na página 30).

[Git24e] Github. *GithubFastApi1*. Consultado em 2024/02/20. 2024. URL: https://github.com/VascoGomesWork/Projecto-IS/tree/main/Code/OnRender/FastApi (citado na página 32).

[Git24f] Github. *GithubMosquitto*. Consultado em 2024/02/20. 2024. URL: https://github.com/VascoGomesWork/Projecto-IS/tree/main/Code/RPi/code/mosquitto (citado nas páginas 30–32).

[Goo24a] Google. *Firebase*. Consultado em 2024/02/20. 2024. URL: https://firebase.google.com/ (citado nas páginas 10, 11).

[Goo24b] Google. *Vision*. Consultado em 2024/02/20. 2024. URL: https://cloud.google.com/vision (citado nas páginas 8, 9).

[Han24] Handsontec. *MFRC522*. Consultado em 2024/02/20. 2024. URL: https://www.handsontec.com/dataspecs/RC522.pdf (citado nas páginas 19, 32).

[IBM24] IBM. Consultado em 2024/02/20. 2024. URL: https://www.ibm.com/topics/internet-of-things (citado na página 31).

[Int24] Interaction. *Cognitive*. Consultado em 2024/02/20. 2024. URL: https://www.interaction-design.org/literature/article/how-to-conduct-a-cognitive-walkthrough (citado na página 35).

[Kas24] Kaspersky. *IP*. Consultado em 2024/02/20. 2024. URL: https://www.kaspersky.com/resource-center/definitions/what-is-an-ip-address (citado na página 33).

[Mic24] Microsoft. *Cloud*. Consultado em 2024/02/20. 2024. URL: https://azure.microsoft.com/pt-pt/resources/cloud-computing-dictionary/what-is-the-cloud/ (citado nas páginas 4, 6, 8, 9, 30).

[Mon23] MongoDB. *MongoDB CRUD Read*. Consultado em 2023/12/04. 2023. URL: https://www.mongodb.com/docs/mongodb-shell/crud/read/ (citado na página 31).

[Mon24a] MongoDB. Consultado em 2024/02/20. 2024. URL: https://www.mongodb.com/docs/mongodb-shell/reference/data-types/ (citado na página 23).

[Mon24b] MongoDB. Consultado em 2024/02/20. 2024. URL: https://www.mongodb.com/ (citado nas páginas 28, 30, 31).

[Mos24] Mosquitto. *Mosquitto*. Consultado em 2024/02/20. 2024. URL: https://mosquitto.org/ (citado nas páginas 28, 30, 33).

[Nat23] NatGeo. *GPS*. Consultado em 2023/11/17. 2023. URL: https://education.nationalgeographic.org/resource/gps/ (citado nas páginas 4, 6).

[OWA24] OWASP. *OWASP*. Consultado em 2024/02/20. 2024. URL: https://owasp.org/www-project-top-ten/ (citado na página 38).

[Par23a] Parkopedia Parking. *Parkopedia Parking*. Consultado em 2023/11/17. 2023. URL: https://www.parkopedia.com/about-us/ (citado nas páginas 3, 4).

[Par23b] Parkopedia Parking. *Parkopedia Parking Interfaces Gráficas*. Consultado em 2023/11/17. 2023. URL: https://www.makeuseof.com/parkopedia-search-parking-spots-efficiently/ (citado na página 5).

[Par23c] Parkwhiz. *Parkwhiz*. Consultado em 2023/11/17. 2023. URL: https://www.parkwhiz.com/ (citado nas páginas 3, 5).

[Par23d] Parkwhiz. *Parkwhiz Interfaces Gráficas*. Consultado em 2023/11/17. 2023. URL: https://medium.com/@corresrachel/parkwhiz-a-design-review-98c15033af36 (citado na página 6).

[Par23e] Parkwhiz. *Parkwhiz Tarefas Executadas pelos Utilizadores*. Consultado em 2023/11/17. 2023. URL: https://www.parkwhiz.com/how-it-works/ (citado na página 6).

[Pey+21] Md Peyal et al. «IoT Based Cost Effective Car Parking Management for Urban Area». Em: set. de 2021, pp. 70–75. DOI: 10.1109/ISAMSR53229.2021.9567826 (citado nas páginas 3, 10).

[PyM24] PyMongo. *PyMongo*. Consultado em 2023/12/04. 2024. URL: https://pymongo.readthedocs.io/en/stable/ (citado na página 31).

[Pyt24a] Python. *PahoMqtt*. Consultado em 2024/02/20. 2024. URL: https://pypi.org/project/paho-mqtt/ (citado nas páginas 30, 31).

[Pyt24b] Python. *Python*. Consultado em 2024/02/20. 2024. URL: https://www.python.org/ (citado nas páginas 30, 31).

[Ras24a] Raspberrypi. Consultado em 2024/02/20. 2024. URL: https://www.raspberrypi.com/products/raspberry-pi-3-model-b/ (citado nas páginas 8, 9, 28, 30–33).

[Ras24b] Raspberrypi. *RBPiCamera*. Consultado em 2024/02/20. 2024. URL: https://www.raspberrypi.com/documentation/accessories/camera.html (citado nas páginas 8, 20, 28, 31).

[Rea24] Realpars. *ServoMotor*. Consultado em 2024/02/20. 2024. URL: https://www.realpars.com/blog/servo-motor (citado nas páginas 10, 28, 32).

[Red24] Redhat. *Rest*. Consultado em 2024/02/20. 2024. URL: https://www.redhat.com/en/topics/api/what-is-a-rest-api (citado nas páginas 25, 28).

[Rob24] Roboflow. *Roboflow*. Consultado em 2024/02/20. 2024. URL: https://roboflow.com/ (citado nas páginas 28, 31, 32).

[Swa24] Swagger. *Bearer*. Consultado em 2024/02/20. 2024. URL: https://swagger.io/docs/specification/authentication/bearer-authentication/ (citado na página 29).

[Tab24] Tableau. Consultado em 2024/02/20. 2024. URL: https://www.tableau.com/learn/articles/dashboards/what-is (citado na página 15).

[Tec24] Techtarget. *RFID*. Consultado em 2024/02/20. 2024. URL: https://www.techtarget.com/iotagenda/definition/RFID-radio-frequency-identification (citado nas páginas 19, 28, 32).

[Tow24] Towerpro. *SG-90*. Consultado em 2024/02/20. 2024. URL: https://www.towerpro.com.tw/product/sg90-7/ (citado nas páginas 19, 28).

[Wir24] Wiregard. *WireGuard*. Consultado em 2023/12/04. 2024. URL: https://www.wireguard.com/ (citado na página 33).

[Zip23] Zipitwireless. *IoTSensors*. Consultado em 2023/11/17. 2023. URL: https://www.zipitwireless.com/blog/what-are-iot-sensors-types-uses-and-examples (citado nas páginas 4, 6, 25).

Documento elaborado com base no *template for final reports and dissertations (Instituto Politécnico de Beja)*, disponível em https://www.overleaf.com/project/5d936b9ea273390001434a37, Version 0.9, 2021/12/01, Autor: João Paulo Barros, joao.barros@ipbeja.pt